

UEmployee, Faculty, Staff Reflection Log

Understanding the Problem

I approached the problem by identifying that I needed a base class (UEmployee) to store common information like name and salary, and then create subclasses (Faculty and Staff) with additional details.

At first, I was confused about how subclasses could access the parent class variables. I understood this by learning how `super()` works in constructors.

Code reference (UEmployee class):

```
3 public class UEmployee
4 {
5     private String name;
6     private double salary;
```

Planning the Solution

I broke the problem into steps: create the base class, then extend it into Faculty and Staff, and finally create a test class to display the results.

I chose inheritance so that common attributes like name and salary would not need to be repeated in each class.

```
3 public class Faculty extends UEmployee
3 public class Staff extends UEmployee
```

Implementation

I first created the UEmployee class with variables and getter methods. Then I created the subclasses with their own variables like department and job title. Finally, I tested everything in the University class.

I tested the program by printing the objects and checking if all values displayed correctly.

Code reference (University class):

```
5 public static void main(String[] args)
6 {
7     Faculty f = new Faculty("Hamelton Smith", 90000, "Computer Science");
8     Staff s = new Staff("john Doe", 50000, "Administrator");
9 }
```

Overcoming Challenges

The most difficult part was understanding how to properly use `super()` to pass values from the subclass to the parent class. I also needed to make sure the `toString()` method included both parent and child information.

I solved this by reviewing examples and testing each class separately.

```
7 public Faculty(String n, double s, String dept)
8 {
9     super(n, s);
10    department = dept;
11 }
```

Learning

I learned how inheritance helps organize code and avoid repetition. I also improved my understanding of constructors, method overriding, and using `toString()`.

```
18 public String toString()
19 {
20     return super.toString() + ", Faculty, Department: " + department;
21 }
```